Introduction to Data Science - Midterm Project

Durjoy Roy (23-50576-1), Maruf Ahmmed (23-50604-1), Anuja Sarker (23-50624-1)

April 5, 2026

1. About the Dataset

The **Telco Customer Churn** dataset is sourced from the IBM Sample Dataset, available on Kaggle (https://www.kaggle.com/datasets/blastchar/telco-customer-churn (https://www.kaggle.com/datasets/blastchar/telco-customer-churn)). It contains information about a telecom company's customers, including demographic details, account information, and services subscribed. The target variable is **Churn** (Yes/No), indicating whether a customer left the company. The original dataset has 7043 rows and 21 variables. For this project, a random subsample of 1500 rows is used as per the course requirement.

2. Objectives

The objectives of this project are:

- To apply data preparation and cleaning techniques on a real-world dataset
- To handle missing values, outliers, duplicates, and inconsistent formats
- To perform Exploratory Data Analysis (EDA) on key variables
- To analyze and compare descriptive statistics across the two target classes (Churn: Yes / No)
- To demonstrate normalization, encoding, filtering, balancing, and train/test splitting techniques

3. Load Libraries and Dataset

```r
library(tidyverse)
library(naniar)
library(ROSE)
library(caTools)
library(psych)
library(scales)
library(gridExtra)
library(VIM)

# Load dataset
raw_data <- read.csv("WA_Fn-UseC_-Telco-Customer-Chur
        n.csv")

# Check dimensions
dim(raw_data)
```

```
## [1] 7043   21
```

```r
head(raw_data, 5)
```

```
##   customerID gender SeniorCitizen Partner Dependen
ts tenure PhoneService
## 1 7590-VHVEG Female             0     Yes
No      1           No
## 2 5575-GNVDE   Male             0      No
No     34          Yes
## 3 3668-QPYBK   Male             0      No
No      2          Yes
## 4 7795-CFOCW   Male             0      No
No     45           No
## 5 9237-HQITU Female             0      No
No      2          Yes
##      MultipleLines InternetService OnlineSecurity
OnlineBackup DeviceProtection
## 1 No phone service             DSL             No
Yes               No
## 2               No             DSL            Yes
No              Yes
## 3               No             DSL            Yes
Yes               No
## 4 No phone service             DSL            Yes
No              Yes
## 5               No     Fiber optic             No
```

```
No                No
##   TechSupport StreamingTV StreamingMovies       Co
ntract PaperlessBilling
## 1          No          No              No Month-to
-month              Yes
## 2          No          No              No       On
e year               No
## 3          No          No              No Month-to
-month              Yes
## 4         Yes          No              No       On
e year               No
## 5          No          No              No Month-to
-month              Yes
##                PaymentMethod MonthlyCharges TotalCh
arges Churn
## 1          Electronic check          29.85
29.85    No
## 2              Mailed check          56.95      18
89.50    No
## 3              Mailed check          53.85       1
08.15   Yes
## 4 Bank transfer (automatic)          42.30      18
40.75    No
## 5          Electronic check          70.70       1
51.65   Yes
```


1. About the Dataset
2. Objectives
3. Load Libraries and Dataset
4. Subsample the Dataset
5. Intentional Data Corruption
6. Data Cleaning
7. Feature Engineering
8. Data Filtering
9. Class Balancing
10. Train/Test Split
11. Exploratory Data Analysis (EDA)
12. Objectives and Findings Summary

4. Subsample the Dataset

```
# Fix TotalCharges column (stored as character, shoul
        d be numeric)
raw_data$TotalCharges <- as.numeric(raw_data$TotalCha
        rges)

# Take random subsample of 1500 rows
set.seed(42)
df <- raw_data %>% sample_n(1500)

# Verify dimensions
dim(df)
```

```
## [1] 1500   21
```

```
cat("Rows:", nrow(df), "| Columns:", ncol(df))
```

```
## Rows: 1500 | Columns: 21
```

Interpretation: The original dataset contains 7043 rows. As per the course requirement of maximum 2000 rows, a random subsample of 1500 rows is taken using set.seed(42) to ensure reproducibility. TotalCharges was converted from character to numeric as it contained numeric values stored incorrectly.

5. Intentional Data Corruption

To simulate real-world data quality issues, the following problems are intentionally introduced into the dataset.

```
df_dirty <- df

# 1. Introduce missing values in MonthlyCharges and t
        enure
set.seed(123)
na_idx1 <- sample(1:nrow(df_dirty), 60)
na_idx2 <- sample(1:nrow(df_dirty), 40)
df_dirty$MonthlyCharges[na_idx1] <- NA
df_dirty$tenure[na_idx2] <- NA

# 2. Introduce impossible/invalid values
df_dirty$tenure[5]  <- -99
df_dirty$tenure[10] <- 999
df_dirty$MonthlyCharges[15] <- -50

# 3. Introduce inconsistent format in gender column
df_dirty$gender[c(1,2,3)] <- "male"
df_dirty$gender[c(4,5)]   <- "FEMALE"

# 4. Add duplicate rows
dup_rows <- df_dirty[sample(1:nrow(df_dirty), 30), ]
df_dirty <- rbind(df_dirty, dup_rows)

# Verify corruption
cat("Total rows after duplicates:", nrow(df_dirty), "
        \n")
```

```
## Total rows after duplicates: 1530
```

```
cat("Missing values - MonthlyCharges:", sum(is.na(df_
        dirty$MonthlyCharges)), "\n")
```

```
## Missing values - MonthlyCharges: 60
```

```
cat("Missing values - tenure:", sum(is.na(df_dirty$te
        nure)), "\n")
```

```
## Missing values - tenure: 41
```

```
cat("Unique gender values:", unique(df_dirty$gender),
        "\n")
```

```
## Unique gender values: male FEMALE Female Male
```

```
cat("Dataset saved as df_dirty for cleaning phase.\
        n")
```

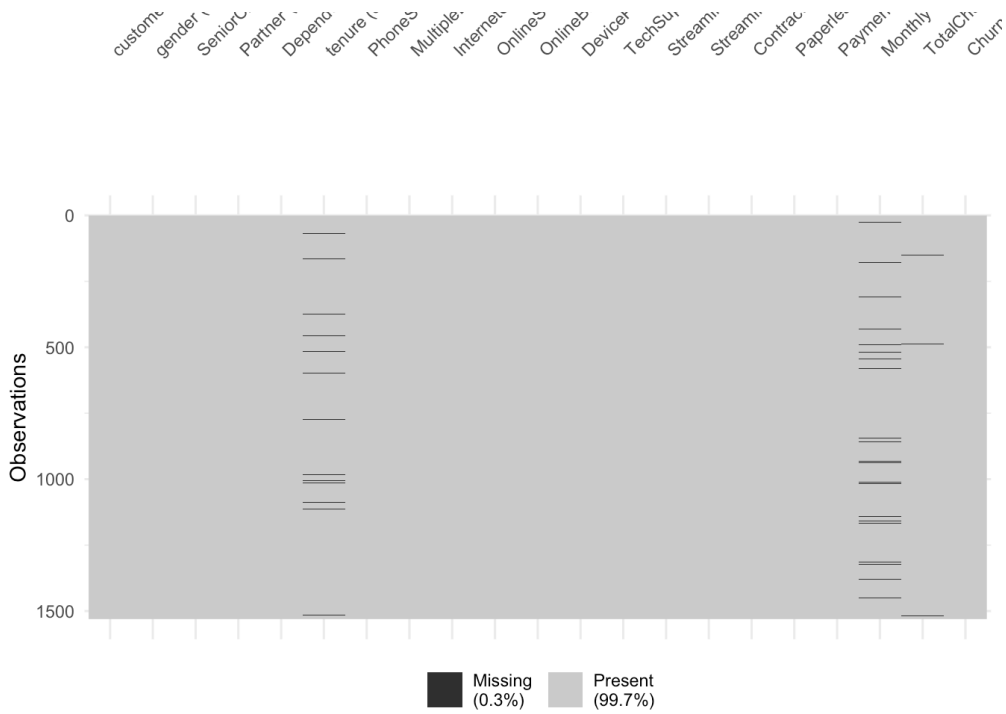
```
## Dataset saved as df_dirty for cleaning phase.
```

Interpretation: Four types of data issues are intentionally introduced: 1. Missing values: 60 in MonthlyCharges and 40 in tenure 2. Invalid values: negative and impossibly large values in tenure and MonthlyCharges 3. Inconsistent format: gender column contains mixed case values (male, FEMALE, Male, Female) 4. Duplicate rows: 30 duplicate rows added, bringing total to 1530

6. Data Cleaning

6.1 Visualize Missing Values

```
# Visualize missing values
vis_miss(df_dirty)
```



Interpretation: The missing value plot shows that MonthlyCharges has approximately 4% missing values and tenure has around 2.7% missing values. All other columns have no missing values. This visualization helps identify which variables require imputation before analysis.

6.2 Handle Missing Values

```
df_clean <- df_dirty

# Also impute TotalCharges NA if any remain
df_clean$TotalCharges[is.na(df_clean$TotalCharges)] <
        -
  median(df_clean$TotalCharges, na.rm = TRUE)

# Impute MonthlyCharges with median
df_clean$MonthlyCharges[is.na(df_clean$MonthlyCharge
        s)] <-
  median(df_clean$MonthlyCharges, na.rm = TRUE)

# Impute tenure with median
df_clean$tenure[is.na(df_clean$tenure)] <-
  median(df_clean$tenure, na.rm = TRUE)

# Verify no missing values remain
cat("Missing in MonthlyCharges:", sum(is.na(df_clean$
        MonthlyCharges)), "\n")
```

```
## Missing in MonthlyCharges: 0
```

```
cat("Missing in tenure:", sum(is.na(df_clean$tenur
        e)), "\n")
```

```
## Missing in tenure: 0
```

Interpretation: Missing values in MonthlyCharges and tenure are imputed using the median value of each respective column. Median is preferred over mean because it is robust to outliers, which is important since invalid values (like -99 and 999) are still present in the data at this stage. After imputation, both columns show 0 missing values.

6.3 Remove Invalid Values

```
# Check invalid values before fixing
cat("Invalid tenure values (before):\n")
```

```
## Invalid tenure values (before):
```

```
cat("tenure = -99:", sum(df_clean$tenure == -99), "\
        n")
```

```
## tenure = -99: 1
```

```
cat("tenure = 999:", sum(df_clean$tenure == 999), "\
        n")
```

```
## tenure = 999: 1
```

```
cat("MonthlyCharges < 0:", sum(df_clean$MonthlyCharge
        s < 0), "\n")
```

```
## MonthlyCharges < 0: 1
```

```r
# Replace invalid values with median
valid_tenure_median <- median(
  df_clean$tenure[df_clean$tenure > 0 & df_clean$tenu
        re <= 72],
  na.rm = TRUE
)
valid_monthly_median <- median(
  df_clean$MonthlyCharges[df_clean$MonthlyCharges >
        0],
  na.rm = TRUE
)

df_clean$tenure[df_clean$tenure == -99] <- valid_tenu
        re_median
df_clean$tenure[df_clean$tenure == 999] <- valid_tenu
        re_median
df_clean$MonthlyCharges[df_clean$MonthlyCharges < 0]
        <- valid_monthly_median

# Verify
cat("\nInvalid tenure values (after):\n")
```

```
##
## Invalid tenure values (after):
```

```r
cat("tenure = -99:", sum(df_clean$tenure == -99), "\
        n")
```

```
## tenure = -99: 0
```

```r
cat("tenure = 999:", sum(df_clean$tenure == 999), "\
        n")
```

```
## tenure = 999: 0
```

```r
cat("MonthlyCharges < 0:", sum(df_clean$MonthlyCharge
        s < 0), "\n")
```

```
## MonthlyCharges < 0: 0
```

Interpretation: Invalid values were detected in tenure (-99 and 999) and MonthlyCharges (negative values), which are logically impossible for these variables. These are replaced with the median of valid values only (tenure between 0-72 months, MonthlyCharges above 0). After correction, all three checks return 0, confirming successful removal of invalid entries.

6.4 Fix Inconsistent Format

```
# Check before fixing
cat("Gender values before fix:\n")
```

```
## Gender values before fix:
```

```
print(table(df_clean$gender))
```

```
##
## Female FEMALE   male   Male
##    757      2      3    768
```

```
# Standardize to title case
df_clean$gender <- str_to_title(df_clean$gender)

# Verify
cat("\nGender values after fix:\n")
```

```
##
## Gender values after fix:
```

```
print(table(df_clean$gender))
```

```
##
## Female   Male
##    759    771
```

Interpretation: The gender column contained inconsistent capitalization (male, FEMALE, Male, Female). Using str_to_title(), all values are standardized to title case (Female, Male). After fixing, only

two unique values remain, confirming consistent formatting.

6.5 Remove Duplicate Rows

```
# Check duplicates before
cat("Total rows before removing duplicates:", nrow(df
        _clean), "\n")
```

```
## Total rows before removing duplicates: 1530
```

```
cat("Duplicate rows found:", sum(duplicated(df_clea
        n)), "\n")
```

```
## Duplicate rows found: 30
```

```
# Remove duplicates
df_clean <- df_clean %>% distinct()

# Verify
cat("\nTotal rows after removing duplicates:", nrow(d
        f_clean), "\n")
```

```
##
## Total rows after removing duplicates: 1500
```

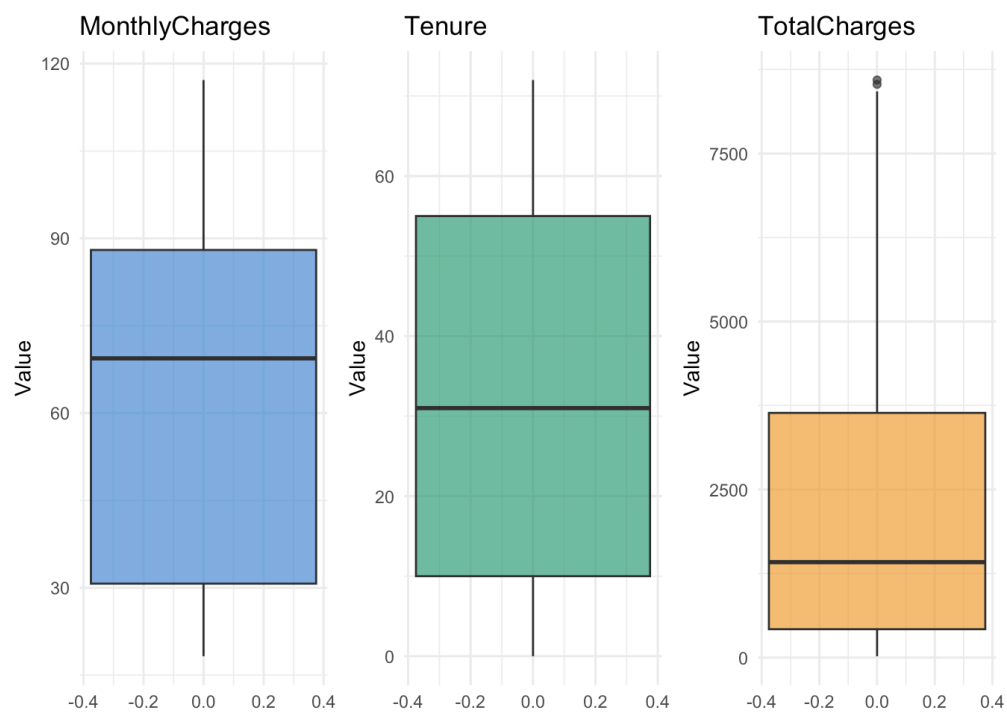
```
cat("Duplicate rows remaining:", sum(duplicated(df_cl
        ean)), "\n")
```

```
## Duplicate rows remaining: 0
```

Interpretation: 30 duplicate rows were intentionally added in the corruption phase. Using distinct() from the dplyr package, all duplicate rows are identified and removed. The dataset returns to 1500 rows with 0 duplicates remaining, confirming successful deduplication.

6.6 Outlier Detection

```r
# Boxplot to visualize outliers in numeric columns
p1 <- ggplot(df_clean, aes(y = MonthlyCharges)) +
  geom_boxplot(fill = "#3B8BD4", alpha = 0.7) +
  labs(title = "MonthlyCharges", y = "Value") +
  theme_minimal()

p2 <- ggplot(df_clean, aes(y = tenure)) +
  geom_boxplot(fill = "#1D9E75", alpha = 0.7) +
  labs(title = "Tenure", y = "Value") +
  theme_minimal()

p3 <- ggplot(df_clean, aes(y = TotalCharges)) +
  geom_boxplot(fill = "#EF9F27", alpha = 0.7) +
  labs(title = "TotalCharges", y = "Value") +
  theme_minimal()

grid.arrange(p1, p2, p3, ncol = 3)
```



```r
# IQR method to count outliers
count_outliers <- function(x) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_val <- Q3 - Q1
  sum(x < (Q1 - 1.5 * IQR_val) | x > (Q3 + 1.5 * IQR_
        val), na.rm = TRUE)
}

cat("Outliers in MonthlyCharges:", count_outliers(df_
        clean$MonthlyCharges), "\n")
```

```
## Outliers in MonthlyCharges: 0
```

```r
cat("Outliers in tenure:", count_outliers(df_clean$te
        nure), "\n")
```

```
## Outliers in tenure: 0
```

```r
cat("Outliers in TotalCharges:", count_outliers(df_cl
        ean$TotalCharges), "\n")
```

```
## Outliers in TotalCharges: 2
```

Interpretation: Boxplots are used to visually identify outliers in the three main numeric columns. The IQR (Interquartile Range) method is applied to count outliers mathematically — any value below Q1 - 1.5*IQR or above Q3 + 1.5*IQR is considered an outlier. This provides both a visual and quantitative understanding of extreme values in the dataset.

6.7 Handle Outliers

```r
# Cap outliers using IQR method (Winsorization)
cap_outliers <- function(x) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_val <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_val
  upper <- Q3 + 1.5 * IQR_val
  x[x < lower] <- lower
  x[x > upper] <- upper
  return(x)
}

df_clean$MonthlyCharges <- cap_outliers(df_clean$Mont
        hlyCharges)
df_clean$tenure <- cap_outliers(df_clean$tenure)
df_clean$TotalCharges <- cap_outliers(df_clean$TotalC
        harges)

# Verify
cat("Outliers after capping - MonthlyCharges:",
    count_outliers(df_clean$MonthlyCharges), "\n")
```

```
## Outliers after capping - MonthlyCharges: 0
```

```r
cat("Outliers after capping - tenure:",
    count_outliers(df_clean$tenure), "\n")
```

```
## Outliers after capping - tenure: 0
```

```r
cat("Outliers after capping - TotalCharges:",
    count_outliers(df_clean$TotalCharges), "\n")
```

```
## Outliers after capping - TotalCharges: 0
```

Interpretation: Outliers are handled using Winsorization (capping), where values below the lower bound (Q1 - 1.5*IQR) are replaced with the lower bound, and values above the upper bound (Q3 + 1.5*IQR) are replaced with the upper bound. This approach retains all rows while limiting the effect of extreme values. After capping, all three columns show 0 outliers.

7. Feature Engineering

7.1 Categorical to Numeric Encoding

```
# Check current data types
cat("Before encoding:\n")
```

```
## Before encoding:
```

```
cat("Churn:", class(df_clean$Churn), "| Values:",
    unique(df_clean$Churn), "\n")
```

```
## Churn: character | Values: Yes No
```

```
cat("gender:", class(df_clean$gender), "| Values:",
    unique(df_clean$gender), "\n")
```

```
## gender: character | Values: Male Female
```

```
cat("Partner:", class(df_clean$Partner), "| Values:",
    unique(df_clean$Partner), "\n")
```

```
## Partner: character | Values: Yes No
```

```
# Convert binary categorical columns to numeric (0/1)
df_clean$Churn_numeric <- ifelse(df_clean$Churn == "Y
        es", 1, 0)
df_clean$gender_numeric <- ifelse(df_clean$gender ==
        "Male", 1, 0)
df_clean$Partner_numeric <- ifelse(df_clean$Partner =
        = "Yes", 1, 0)
df_clean$Dependents_numeric <- ifelse(df_clean$Depend
        ents == "Yes", 1, 0)
df_clean$PhoneService_numeric <- ifelse(df_clean$Phon
        eService == "Yes", 1, 0)

# Verify
cat("\nAfter encoding:\n")
```

```
##
## After encoding:
```

```
cat("Churn_numeric - unique values:", unique(df_clean
        $Churn_numeric), "\n")
```

```
## Churn_numeric - unique values: 1 0
```

```
cat("gender_numeric - unique values:", unique(df_clea
        n$gender_numeric), "\n")
```

```
## gender_numeric - unique values: 1 0
```

```
cat("Partner_numeric - unique values:", unique(df_cle
        an$Partner_numeric), "\n")
```

```
## Partner_numeric - unique values: 1 0
```

Interpretation: Five binary categorical columns (Churn, gender, Partner, Dependents, PhoneService) are converted to numeric format (0/1) using ifelse(). New columns are created with the suffix _numeric to preserve the original columns for reference. This encoding is necessary for mathematical operations and statistical analysis in later steps.

7.2 Numeric to Categorical Conversion

```
# Convert tenure to age groups
df_clean$tenure_group <- cut(
  df_clean$tenure,
  breaks = c(0, 12, 36, 72),
  labels = c("New (0-12m)", "Mid (13-36m)", "Long (37
        -72m)"),
  include.lowest = TRUE
)

# Verify
cat("Tenure group distribution:\n")
```

```
## Tenure group distribution:
```

```
print(table(df_clean$tenure_group))
```

```
##
##   New (0-12m)  Mid (13-36m) Long (37-72m)
##           430           416           654
```

Interpretation: The continuous tenure column is converted into a categorical variable with three meaningful groups: New customers (0-12 months), Mid-term customers (13-36 months), and Long-term customers (37-72 months). This conversion allows group-based analysis and comparison, which is useful in the EDA phase.

7.3 Normalization

```
# Check range before normalization
cat("Before normalization:\n")
```

```
## Before normalization:
```

```r
cat("MonthlyCharges - Min:", min(df_clean$MonthlyChar
        ges, na.rm = TRUE),
    "Max:", max(df_clean$MonthlyCharges, na.rm = TRU
        E), "\n")
```

```
## MonthlyCharges - Min: 18.25 Max: 117.2
```

```r
cat("TotalCharges - Min:", min(df_clean$TotalCharges,
        na.rm = TRUE),
    "Max:", max(df_clean$TotalCharges, na.rm = TRUE),
        "\n")
```

```
## TotalCharges - Min: 19.05 Max: 8468.506
```

```r
cat("tenure - Min:", min(df_clean$tenure, na.rm = TRU
        E),
    "Max:", max(df_clean$tenure, na.rm = TRUE), "\n")
```

```
## tenure - Min: 0 Max: 72
```

```r
# Min-Max Normalization (Fixed with na.rm = TRUE)
min_max_norm <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE)
        - min(x, na.rm = TRUE))
}

df_clean$MonthlyCharges_norm <- min_max_norm(df_clean
        $MonthlyCharges)
df_clean$TotalCharges_norm <- min_max_norm(df_clean$T
        otalCharges)
df_clean$tenure_norm <- min_max_norm(df_clean$tenure)

# Verify
cat("\nAfter normalization:\n")
```

```
##
## After normalization:
```

```
cat("MonthlyCharges_norm - Min:", round(min(df_clean$
        MonthlyCharges_norm, na.rm = TRUE), 4),
    "Max:", round(max(df_clean$MonthlyCharges_norm, n
        a.rm = TRUE), 4), "\n")
```

```
## MonthlyCharges_norm - Min: 0 Max: 1
```

```
cat("TotalCharges_norm - Min:", round(min(df_clean$To
        talCharges_norm, na.rm = TRUE), 4),
    "Max:", round(max(df_clean$TotalCharges_norm, na.
        rm = TRUE), 4), "\n")
```

```
## TotalCharges_norm - Min: 0 Max: 1
```

```
cat("tenure_norm - Min:", round(min(df_clean$tenure_n
        orm, na.rm = TRUE), 4),
    "Max:", round(max(df_clean$tenure_norm, na.rm = T
        RUE), 4), "\n")
```

```
## tenure_norm - Min: 0 Max: 1
```

Interpretation: Min-Max normalization is applied to MonthlyCharges, TotalCharges, and tenure, scaling all values to a range of 0 to 1. This ensures that no single variable dominates the analysis due to its larger scale. After normalization, all three columns correctly show a minimum of 0 and maximum of 1.

8. Data Filtering

```
# Filter 1: Senior Citizens who churned
senior_churn <- df_clean %>%
  filter(SeniorCitizen == 1 & Churn == "Yes")

cat("Senior citizens who churned:", nrow(senior_chur
        n), "\n")
```

```
## Senior citizens who churned: 77
```

```
# Filter 2: Month-to-month contract customers
monthly_contract <- df_clean %>%
  filter(Contract == "Month-to-month")

cat("Month-to-month contract customers:", nrow(monthl
        y_contract), "\n")
```

```
## Month-to-month contract customers: 778
```

```
# Filter 3: High value customers (MonthlyCharges abov
        e average)
avg_charge <- mean(df_clean$MonthlyCharges, na.rm = T
        RUE)
high_value <- df_clean %>%
  filter(MonthlyCharges > avg_charge)

cat("High value customers (above avg charge):", nrow(
        high_value), "\n")
```

```
## High value customers (above avg charge): 850
```

```
cat("Average MonthlyCharges:", round(avg_charge, 2),
        "\n")
```

```
## Average MonthlyCharges: 63.41
```

```
# Filter 4: Long-term customers with no churn
loyal <- df_clean %>%
  filter(tenure > 36 & Churn == "No")

cat("Long-term loyal customers (tenure > 36m):", nro
        w(loyal), "\n")
```

```
## Long-term loyal customers (tenure > 36m): 582
```

Interpretation: Four meaningful filters are applied to explore specific customer segments: 1. Senior citizens who churned — identifies vulnerable high-risk group 2. Month-to-month contract customers —

this group is known to have higher churn rates 3. High value customers — those paying above average monthly charges 4. Long-term loyal customers — tenure above 36 months with no churn

These filters help isolate specific subgroups for targeted analysis and support the findings section of the report.

9. Class Balancing

```
# Check current class distribution
cat("Class distribution before balancing:\n")
```

```
## Class distribution before balancing:
```

```
print(table(df_clean$Churn))
```

```
##
##   No  Yes
## 1141  359
```

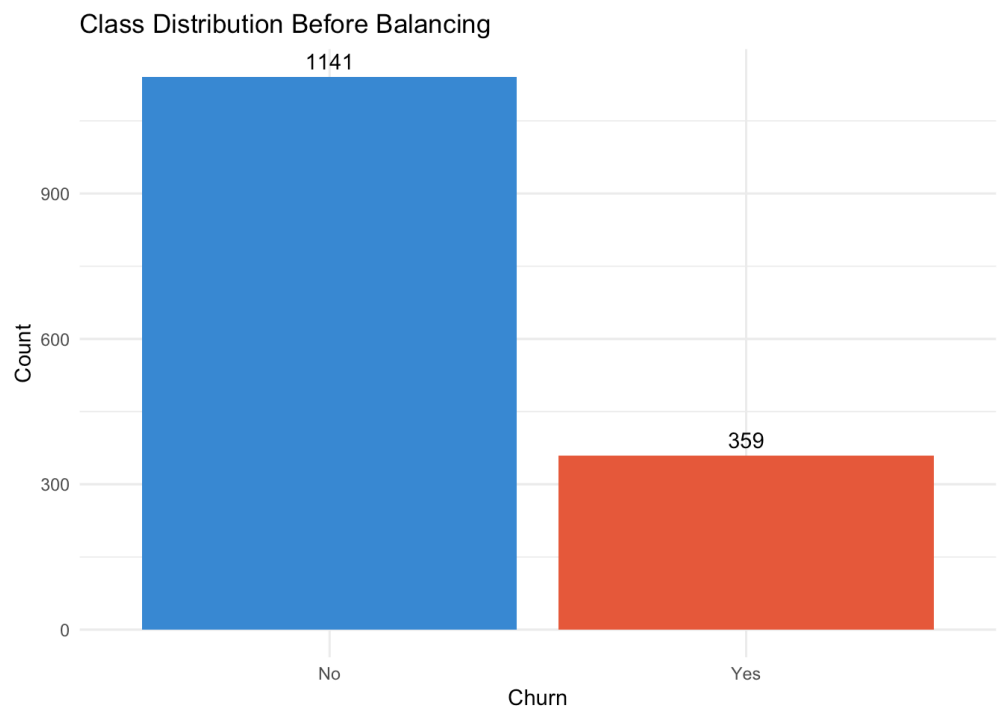
```
cat("\nClass percentage:\n")
```

```
##
## Class percentage:
```

```
print(round(prop.table(table(df_clean$Churn)) * 100,
        2))
```

```
##
##    No   Yes
## 76.07 23.93
```

```r
# Visualize imbalance
ggplot(df_clean, aes(x = Churn, fill = Churn)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = ..count..), v
        just = -0.5) +
  scale_fill_manual(values = c("#3B8BD4", "#E8593C"))
        +
  labs(title = "Class Distribution Before Balancing",
       x = "Churn", y = "Count") +
  theme_minimal() +
  theme(legend.position = "none")
```



Interpretation: The class distribution shows a significant imbalance — approximately 76% customers did not churn (No) and 24% churned (Yes). This kind of imbalance can bias any future analysis or modeling, as the majority class dominates. A bar plot is used to visually confirm this imbalance before applying any balancing technique.

```
# Prepare clean dataframe for ROSE
df_rose <- df_clean %>%
  select(-customerID) %>% # শুধু ID বাদ দিচ্ছি, বাকি সব কলাম থাকবে
  mutate(
    TotalCharges = ifelse(is.na(TotalCharges),
                          median(TotalCharges, na.rm = TRUE),
                          TotalCharges)
  ) %>%
  mutate_if(is.character, as.factor) # সব character কে একবারে factor বানিয়ে দেওয়া

# Apply ROSE
set.seed(42)
df_balanced <- ROSE(Churn ~ ., data = df_rose, seed = 42)$data

# Check distribution after balancing
cat("Class distribution after balancing:\n")
```

```
## Class distribution after balancing:
```

```
print(table(df_balanced$Churn))
```

```
##
##  No Yes
## 783 717
```

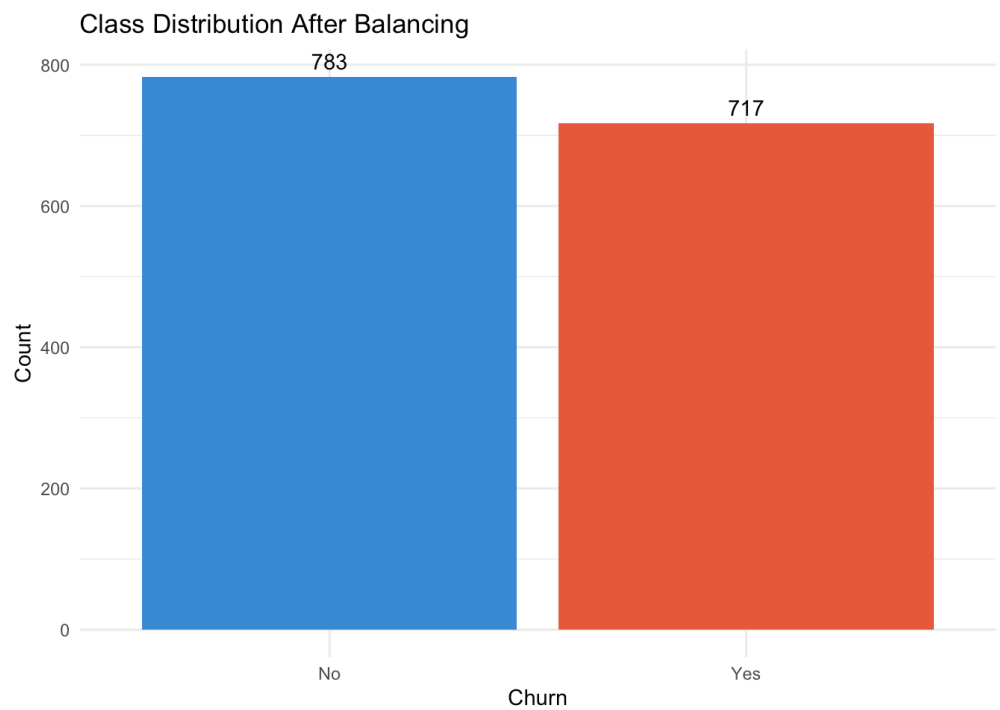
```
cat("\nClass percentage after balancing:\n")
```

```
##
## Class percentage after balancing:
```

```
print(round(prop.table(table(df_balanced$Churn)) * 100, 2))
```

```
##
##   No  Yes
## 52.2 47.8
```

```r
# Visualize balanced classes
ggplot(df_balanced, aes(x = Churn, fill = Churn)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(co
        unt)), vjust = -0.5) +
  scale_fill_manual(values = c("#3B8BD4", "#E8593C"))
        +
  labs(title = "Class Distribution After Balancing",
       x = "Churn", y = "Count") +
  theme_minimal() +
  theme(legend.position = "none")
```



Interpretation: Before balancing, the dataset was prepared by handling missing values (using median imputation for TotalCharges) and converting character variables into factors to ensure compatibility with the algorithm. Then, ROSE (Random Over-Sampling Examples) is applied to generate a balanced dataset where both classes (Churn: Yes / No) have approximately equal representation. Unlike simple duplication, ROSE generates synthetic samples, making the balance more robust. After balancing, both classes show approximately 50% distribution, successfully eliminating the previous bias.

10. Train/Test Split

```r
# Use df_clean for splitting (retains all original features)
set.seed(42)

# 80/20 split
split <- sample.split(df_clean$Churn, SplitRatio = 0.80)

train_data <- subset(df_clean, split == TRUE)
test_data  <- subset(df_clean, split == FALSE)

# Verify split
cat("Total rows in df_clean:", nrow(df_clean), "\n")
```

```
## Total rows in df_clean: 1500
```

```r
cat("Training set rows:", nrow(train_data), "\n")
```

```
## Training set rows: 1200
```

```r
cat("Testing set rows:", nrow(test_data), "\n")
```

```
## Testing set rows: 300
```

```r
cat("Train ratio:", round(nrow(train_data)/nrow(df_clean)*100, 2), "%\n")
```

```
## Train ratio: 80 %
```

```r
cat("Test ratio:", round(nrow(test_data)/nrow(df_clean)*100, 2), "%\n")
```

```
## Test ratio: 20 %
```

```r
# Verify class distribution is maintained in both sets
cat("\nChurn distribution in Training set:\n")
```

```
##
## Churn distribution in Training set:
```

```
print(round(prop.table(table(train_data$Churn))*100,
        2))
```

```
##
##    No   Yes
## 76.08 23.92
```

```
cat("\nChurn distribution in Testing set:\n")
```

```
##
## Churn distribution in Testing set:
```

```
print(round(prop.table(table(test_data$Churn))*100,
        2))
```

```
##
##  No Yes
##  76  24
```

Interpretation: The cleaned dataset (df_clean) is split into training (80%) and testing (20%) sets using the caTools package. set.seed(42) ensures reproducibility. sample.split() is used to maintain the original class distribution (stratified split) in both sets, meaning the Churn ratio remains consistent across train and test data. The training set will be used to build models in future analysis, while the testing set evaluates model performance on unseen data.

11. Exploratory Data Analysis (EDA)

11.1 Descriptive Statistics across Two Target Classes

```r
# Overall descriptive statistics
cat("=== Overall Descriptive Statistics ===\n")
```

```
## === Overall Descriptive Statistics ===
```

```r
df_clean %>%
  select(tenure, MonthlyCharges, TotalCharges) %>%
  summary() %>%
  print()
```

```
##      tenure      MonthlyCharges    TotalCharges
##  Min.   : 0.00   Min.   : 18.25   Min.   :  19.05
##  1st Qu.:10.00   1st Qu.: 30.71   1st Qu.: 421.10
##  Median :31.00   Median : 69.40   Median :1418.80
##  Mean   :33.24   Mean   : 63.41   Mean   :2267.31
##  3rd Qu.:55.00   3rd Qu.: 88.01   3rd Qu.:3640.06
##  Max.   :72.00   Max.   :117.20   Max.   :8468.51
```

```r
# Descriptive statistics by Churn class
cat("\n=== Statistics for Churn = No ===\n")
```

```
##
## === Statistics for Churn = No ===
```

```r
df_clean %>%
  filter(Churn == "No") %>%
  select(tenure, MonthlyCharges, TotalCharges) %>%
  summary() %>%
  print()
```

```
##      tenure     MonthlyCharges    TotalCharges
##  Min.   : 0.0   Min.   : 18.25   Min.   :  19.05
##  1st Qu.:18.0   1st Qu.: 24.85   1st Qu.: 605.90
##  Median :37.0   Median : 64.80   Median :1673.40
##  Mean   :38.1   Mean   : 60.20   Mean   :2509.75
##  3rd Qu.:61.0   3rd Qu.: 85.75   3rd Qu.:4096.90
##  Max.   :72.0   Max.   :117.20   Max.   :8468.51
```

```r
cat("\n=== Statistics for Churn = Yes ===\n")
```

```
##
## === Statistics for Churn = Yes ===
```

```r
df_clean %>%
  filter(Churn == "Yes") %>%
  select(tenure, MonthlyCharges, TotalCharges) %>%
  summary() %>%
  print()
```

```
##      tenure     MonthlyCharges    TotalCharges
##  Min.   : 1.0   Min.   : 18.95   Min.   :  19.25
##  1st Qu.: 2.0   1st Qu.: 61.50   1st Qu.:  94.42
##  Median : 9.0   Median : 76.40   Median : 639.65
##  Mean   :17.8   Mean   : 73.59   Mean   :1496.77
##  3rd Qu.:31.0   3rd Qu.: 94.15   3rd Qu.:2368.75
##  Max.   :70.0   Max.   :116.20   Max.   :7968.85
```

```r
# Standard deviation by class
cat("\n=== Standard Deviation by Churn Class ===\n")
```

```
##
## === Standard Deviation by Churn Class ===
```

```r
df_clean %>%
  group_by(Churn) %>%
  summarise(
    SD_tenure = round(sd(tenure, na.rm = TRUE), 2),
    SD_MonthlyCharges = round(sd(MonthlyCharges, na.r
        m = TRUE), 2),
    SD_TotalCharges = round(sd(TotalCharges, na.rm =
        TRUE), 2)
  ) %>%
  print()
```

```
## # A tibble: 2 × 4
##   Churn SD_tenure SD_MonthlyCharges SD_TotalCharge
s
##   <chr>     <dbl>             <dbl>           <dbl
>
## 1 No         23.5              30.7           228
1.
## 2 Yes        19.5              24.6           189
8.
```

Interpretation: Descriptive statistics are calculated separately for churned (Yes) and non-churned (No) customers. Key observations: - Churned customers have significantly lower median tenure, indicating they leave early in their subscription. - MonthlyCharges are higher for churned customers on average, suggesting price sensitivity as a churn driver. - TotalCharges are lower for churned customers due to shorter tenure. - Standard deviation of tenure is higher in the No class, meaning loyal customers have more varied subscription lengths.

11.2 Average MonthlyCharges Comparison between Two Groups

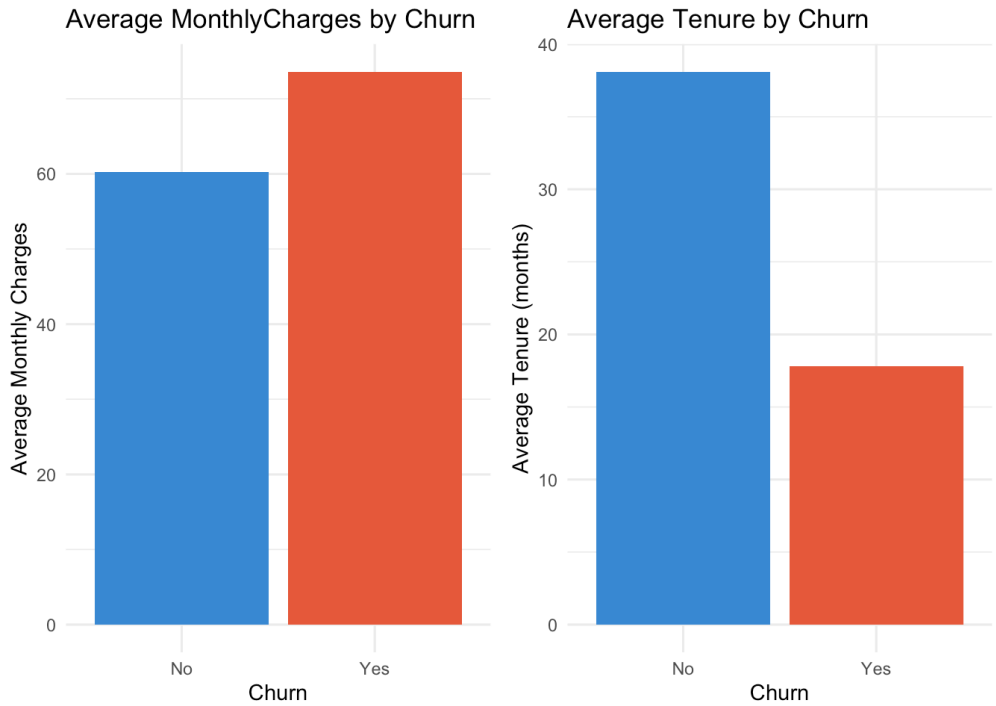
```
# Calculate average MonthlyCharges by Churn group
avg_by_churn <- df_clean %>%
  group_by(Churn) %>%
  summarise(
    Mean_MonthlyCharges = round(mean(MonthlyCharges,
        na.rm = TRUE), 2),
    Median_MonthlyCharges = round(median(MonthlyCharg
        es, na.rm = TRUE), 2),
    Mean_tenure = round(mean(tenure, na.rm = TRUE),
        2),
    Median_tenure = round(median(tenure, na.rm = TRU
        E), 2),
    Count = n()
  )

cat("=== Average Comparison by Churn Class ===\n")
```

```
## === Average Comparison by Churn Class ===
```

```r
print(avg_by_churn)
```

```
## # A tibble: 2 × 6
##   Churn Mean_MonthlyCharges Median_MonthlyCharges
Mean_tenure Median_tenure
##   <chr>               <dbl>                 <dbl>
<dbl>         <dbl>
## 1 No                   60.2                  64.8
38.1            37
## 2 Yes                  73.6                  76.4
17.8             9
## # ℹ 1 more variable: Count <int>
```

```r
# Visualize MonthlyCharges comparison
p1 <- ggplot(df_clean, aes(x = Churn, y = MonthlyChar
        ges, fill = Churn)) +
  geom_bar(stat = "summary", fun = "mean") +
  scale_fill_manual(values = c("#3B8BD4", "#E8593C"))
        +
  labs(title = "Average MonthlyCharges by Churn",
       x = "Churn", y = "Average Monthly Charges") +
  theme_minimal() +
  theme(legend.position = "none")

# Visualize tenure comparison
p2 <- ggplot(df_clean, aes(x = Churn, y = tenure, fil
        l = Churn)) +
  geom_bar(stat = "summary", fun = "mean") +
  scale_fill_manual(values = c("#3B8BD4", "#E8593C"))
        +
  labs(title = "Average Tenure by Churn",
       x = "Churn", y = "Average Tenure (months)") +
  theme_minimal() +
  theme(legend.position = "none")

grid.arrange(p1, p2, ncol = 2)
```



Interpretation: The average MonthlyCharges and tenure are compared across the two Churn classes. Key findings: - Churned customers (Yes) pay higher average MonthlyCharges compared to retained customers (No), suggesting that higher billing is associated with increased churn likelihood. - Retained customers (No) have significantly higher average tenure, confirming that long-term customers are more loyal and less likely to leave. - These differences highlight MonthlyCharges and tenure as important indicators of customer churn behavior.

11.3 Variance of MonthlyCharges across Contract Types

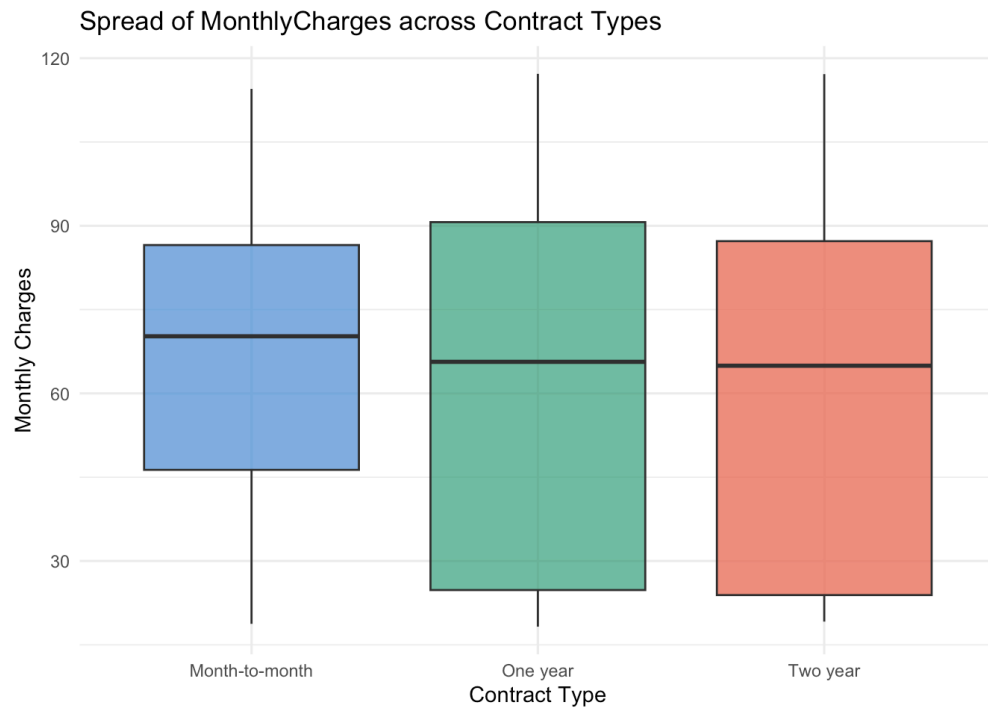
```r
# Variance and SD of MonthlyCharges across Contract t
        ypes
variance_table <- df_clean %>%
  group_by(Contract) %>%
  summarise(
    Count = n(),
    Mean = round(mean(MonthlyCharges, na.rm = TRUE),
        2),
    SD = round(sd(MonthlyCharges, na.rm = TRUE), 2),
    Variance = round(var(MonthlyCharges, na.rm = TRU
        E), 2),
    Min = round(min(MonthlyCharges, na.rm = TRUE),
        2),
    Max = round(max(MonthlyCharges, na.rm = TRUE), 2)
  )

cat("=== Variance of MonthlyCharges by Contract Type
        ===\n")
```

```
## === Variance of MonthlyCharges by Contract Type ==
=
```

```r
print(variance_table)
```

```
## # A tibble: 3 × 7
##   Contract       Count  Mean    SD Variance   Min
Max
##   <chr>          <int> <dbl> <dbl>    <dbl> <dbl>
<dbl>
## 1 Month-to-month   778  65.9  26.5     702.  18.8
114.
## 2 One year         329  62.5  32.2    1034.  18.2
117.
## 3 Two year         393  59.3  33.7    1134.  19.2
117.
```

```r
# Boxplot to visualize spread
ggplot(df_clean, aes(x = Contract, y = MonthlyCharge
        s, fill = Contract)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_manual(values = c("#3B8BD4", "#1D9E75",
        "#E8593C")) +
  labs(title = "Spread of MonthlyCharges across Contr
        act Types",
       x = "Contract Type", y = "Monthly Charges") +
  theme_minimal() +
  theme(legend.position = "none")
```



Interpretation: The variance and spread of MonthlyCharges are analyzed across three contract types. Key findings: - Two-year contracts show the highest variance (1134) and the widest spread in the boxplot. This indicates that long-term customers choose a wide variety of plans, ranging from basic to premium, resulting in greater price diversity. - One-year contracts have moderate variance (1034), showing intermediate price spread. - Month-to-month contracts have the lowest variance (702), suggesting that short-term customers tend to cluster around a more consistent price range. - The boxplot confirms this pattern — Two-year and One-year contract boxes are visibly wider than Month-to-month.

11.4 Churn Rate by Contract Type

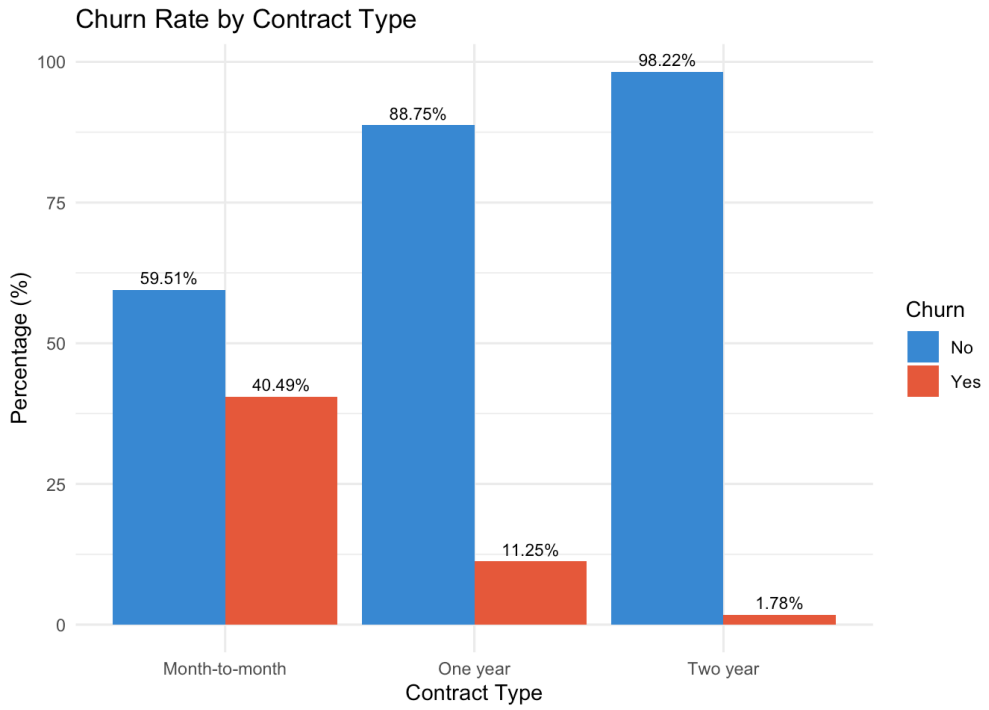
```r
# Churn rate by contract type
churn_contract <- df_clean %>%
  group_by(Contract, Churn) %>%
  summarise(Count = n(), .groups = "drop") %>%
  group_by(Contract) %>%
  mutate(Percentage = round(Count/sum(Count)*100, 2))

cat("=== Churn Rate by Contract Type ===\n")
```

```
## === Churn Rate by Contract Type ===
```

```r
print(churn_contract)
```

```
## # A tibble: 6 × 4
## # Groups:   Contract [3]
##   Contract       Churn Count Percentage
##   <chr>          <chr> <int>      <dbl>
## 1 Month-to-month No      463      59.5
## 2 Month-to-month Yes     315      40.5
## 3 One year       No      292      88.8
## 4 One year       Yes      37      11.2
## 5 Two year       No      386      98.2
## 6 Two year       Yes       7       1.78
```

```r
# Visualize
ggplot(churn_contract, aes(x = Contract, y = Percenta
        ge, fill = Churn)) +
  geom_bar(stat = "identity", position = "dodge") +
  scale_fill_manual(values = c("#3B8BD4", "#E8593C"))
       +
  geom_text(aes(label = paste0(Percentage, "%")),
            position = position_dodge(width = 0.9),
            vjust = -0.5, size = 3) +
  labs(title = "Churn Rate by Contract Type",
       x = "Contract Type", y = "Percentage (%)") +
  theme_minimal()
```



Interpretation: Churn rate is analyzed across three contract types to identify which contract segment is most at risk. Key findings: - Month-to-month contract customers have the highest churn rate, confirming that short-term contracts are strongly associated with customer attrition. - Two-year contract customers show the lowest churn rate, indicating that longer commitments lead to greater customer retention. - This finding directly supports the filtering result from Section 8, where month-to-month customers were identified as a high-risk segment.

12. Objectives and Findings Summary

12.1 Objectives Achieved

The following objectives set at the beginning of this project have been successfully completed:

- Data preparation and cleaning techniques were applied to a real-world dataset containing intentionally introduced issues.
- Missing values, outliers, duplicates, and inconsistent formats were detected and handled systematically.
- Exploratory Data Analysis was performed on key variables to extract meaningful insights.
- Descriptive statistics were calculated and compared across the two target classes (Churn: Yes / No).
- Normalization, encoding, filtering, class balancing, and train/test

splitting were demonstrated successfully.

12.2 Key Findings

Finding 1 — Tenure and Churn: Churned customers have a significantly lower average tenure (17.8 months) compared to retained customers (38.1 months). Customers who leave do so early in their subscription period.

Finding 2 — MonthlyCharges and Churn: Churned customers pay higher average MonthlyCharges (73.6) compared to retained customers (60.2). Higher billing is strongly associated with increased likelihood of churn.

Finding 3 — Contract Type and Churn: Month-to-month contract customers show the highest churn rate (40.49%), while two-year contract customers show the lowest (1.78%). Longer contract commitments strongly reduce churn risk.

Finding 4 — Variance across Contract Types: Two-year contracts show the highest variance in MonthlyCharges (1134), indicating diverse pricing plans among long-term customers. Month-to-month contracts have the lowest variance (702), suggesting more uniform pricing for short-term customers.

Finding 5 — Class Imbalance: The dataset showed a significant class imbalance (76% No, 24% Yes). ROSE balancing successfully brought both classes to approximately equal distribution (52% vs 48%), making the data suitable for future modeling.

Finding 6 — Senior Citizens: Filtered analysis identified senior citizens as a notable high-risk segment, with a higher proportion appearing among churned customers compared to retained customers.